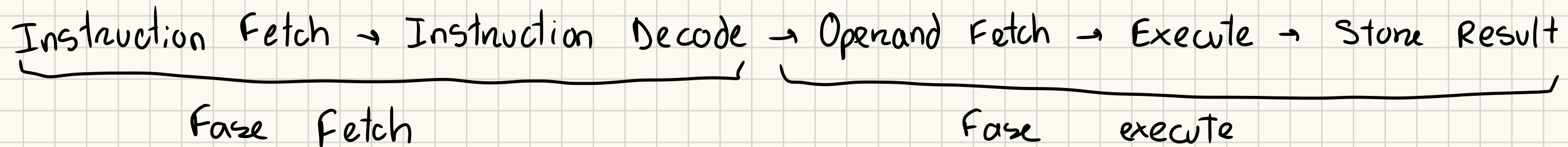


Classes de Instruções

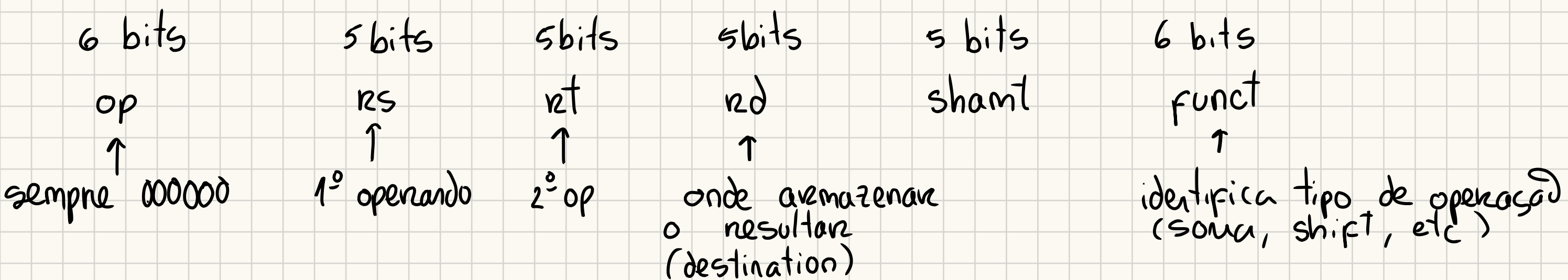
- Processamento (Aritméticas e lógicas)
- Transferência de informação (copia entre registos (internos-internos ou internos-memória)
- Controlo de fluxo (estruturas condicionais, ciclos, chamadas a funções)



Anquitetona Mips

- 32 bits com 32 bits cada um (word = 32 bits - dimensão dos registros)
- Todas as instruções são codificadas com 32 bits
- Memória externa organizada em bytes (byte addressable)
 - ↳ dimensão do menor elemento da memória é 8 bits - 1 byte
 - ↳ Cada byte é um endereço distinto
- Espaço de endereçamento de 32 bits (2^{32} endereços distintos)
- Barramento de dados externo de 32 bits
- Arquitetura load-store (operações aritméticas e lógicas apenas em registros internos)
 - ↳ register - register operation

Instruções do tipo R



- add \$8, \$17, \$18

0	\$17	\$18	\$8	0	32	
000000	10001	10010	01000	00000	100000	= 0x 02324020

- sll \$10, \$16, 4

0	0	\$16	\$10	4	0	
000000	00000	10000	01010	00100	000000	= 0x 00105100

- 0x 00000000 → NOP

Formato I - lw, sw, salto condicional complemento p/ 2

op
6 bits

rs
5 bits

rt
5 bits

offset
16 bits
 ↓
 [-32768, +32767]

10 11 12 13 14
 A B C D E

• lw \$8, 20(\$19)

35 19 8 20
100011 10011 01000 0000 0000 0001 0100 = 0x8E680014

• sw \$8, 20(\$19)

43 19 8 20
101011 10011 01000 0000 0000 0001 0100 = 0x4E680014

• bne \$19, \$20, ELSE → 0x0040000C

↪ 0x40000C - 0x400004 = 0x08 0x08/4 = 0x02

5 19 20 0x002

000101 10011 10100 0000 0000 0000 0010 = 0x16740002

• $0x1509FFFD$ = $\underbrace{0001}_{\text{op code}} \underbrace{0101}_{\text{rs}} \underbrace{0000}_{\text{rt}} \underbrace{1001}_{\text{offset}} \underbrace{1111}_{\text{negativo}} \underbrace{1111}_{\text{negativo}} \underbrace{1111}_{\text{negativo}} \underbrace{1101}_{\text{negativo}}$
 $PC = 00400030$
 $\downarrow$
 $bne \$8, \$9, 0x40028$
 $\underbrace{0000}_{12} \underbrace{1100}_{12} = 0x0C$
 $1111 \ 1111 \ 1111 \ 1101 = -3_{10}$
 $\text{Endereço alvo} = (PC + 4) + (-3 * 4) =$
 $= 0x00400034 - 0x0C$
 $= 0x0040028$

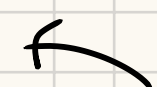
Organização da informação na memória

- Se a quantidade a armazenar tiver mais que 8 bits, ela ocupará várias posições consecutivas
 ↳ Uma word de 32 bits ocupará 4 posições

Ordem de armazenamento?

Ex: $0x012387A5$

big-endian:



Implementado pelo MIPS,
não pelo ISA!

little-endian:

01 - $0x10010000$

23 - $0x10010001$

87 - $0x10010002$

A5 - $0x10010003$

A5 - $0x10010000$

87 - $0x10010001$

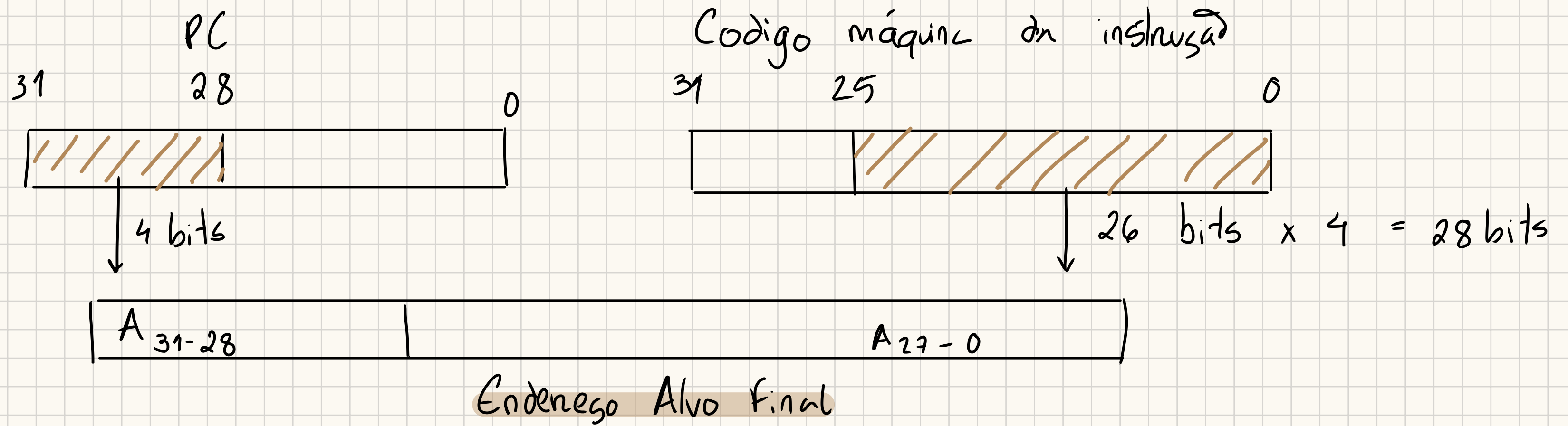
23 - $0x10010002$

01 - $0x10010003$

formato J - j (endereço pseudo-direto)

op
6 bits

28 LSB >> 2 (pq endereço é sempre mult de 4 → 2 bits são 0)
26 bits



JR - Endereçamento indireto por registro

Endereço tem de ser do .text

No formato R:

0 rs 0 0 0 8

Modos de Endereçamento

- aritméticas e lógicas — endereçamento tipo registro
- Acesso à memória — endereçamento indireto por registro com deslocamento
- Salto condicional — endereçamento relativo ao PC
- JR — endereçamento indireto por registro
- J — endereçamento (pseudo) direto
- Aritméticas e lógicas com sufixo "i" — endereçamento imediato

Inteiros

$$11001011 + 00110111 = 1 \text{ } 00000010$$

carry

Se unsigned: carry significa overflow

se signed: carry não indica overflow

Multiplicação

- ↳ feito num bloco à parte
- ↳ resultado de 64 bits (cada operando é de 32)
- ↳ HI para os 32 mais significativos, LO para os restantes

Divisão

- ↳ primeiro é feita em módulo e depois são aplicados os sinais
- ↳ se os sinais do dividendo e divisor forem iguais → positivo
- ↳ resto tem o mesmo sinal do dividendo

$$-7 / 3 = \begin{array}{r} 7 \overline{) 3} \\ -6 \\ \hline 1 \end{array} \quad \text{quociente} = -2, \text{ resto} = -1$$

- Resultado final em 64 bits (resto e quociente)
- ↓ ↓
HI LO

divisor > dividendo → quociente = 0, resto = dividendo

$$4/6 \rightarrow LO = 0, HI = 6$$

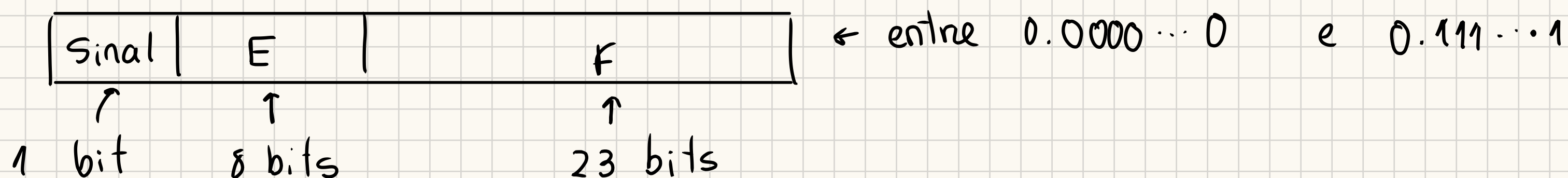
IEEE754

- Representação normalizada $\rightarrow$ um algarismo à esquerda da virgula (diferente de 0)

$$N = (+/-) 1.f \times 2^{\text{Exp}}$$

$\downarrow$ $\leftarrow$ expoente
 $\uparrow$ parte fracionária
mantissa

- Sempre em base 2
- + bits na parte fracionária = maior precisão
- + bits no expoente = maior gama de representação



Como a representação é normalizada o bit à esquerda da virgula é sempre 1 (constante) e por isso não é representado - hidden bit

$$1.00 \dots 0 \leq \text{mantissa} \leq 1.11 \dots 1$$

$$E = \text{Exp verdadeiro} + 127$$

$$N = (-1)^S 1.f \times 2^{E-127}$$

$E = 127$ representa expoente 0
 $E > 127$ representa expoente positivo
 $E < 127$ representa expoente negativo

0 10000010 101100000000000000000000 = 0x41580000
 1
 positivo $\downarrow$ Mantissa = 1.1011... = 1.1011
 130
 $E = 130 - 127 = 3$

Resultado = $+ 1.1011 \times 2^3 = (1 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4}) \times 2^3 = 1.6875 \times 8$
 ou $= 13.5$

$R = + 1.1011 \times 2^3 = 1101.1 \rightarrow i = 13_{10}, F = 0.1 = 0.5_{10} \rightarrow R = 13.5$

-12.59375₁₀

$S = 1$

$i = 12_{10} = 1100_2$

$F = .59375_{10} = .10011_2$

$12.59375 = 1100.10011_2 \times 2^0$
 $= 1.10010011_2 \times 2^3$

$E = 3 + 127 = 130_{10} = 10000010_2$

$\therefore 1\ 1000\ 0010\ 100100110 \dots 0 = 0x\ C1498000$

MSb $\downarrow$

	.59375	
	$\times 2$	
	1.18750	
	.18750	
	$\times 2$	
	0.37500	
	$\times 2$	
	0.75000	
	$\times 2$	
	1.50000	
	.50000	
	$\times 2$	
LSb $\downarrow$	1.00000	

Uma representação decimal tem no máximo 6 casas decimais

Overflow: $E > 254 \rightarrow N_{\text{resultado}} > 1.1 \dots 1 \times 2^{+127}$

Underflow: $E < 1 \rightarrow N_{\text{resultado}} < 1.00 \dots 0 \times 2^{-126}$

Quando $E = 255$ e $f = 0 \rightarrow$ Infinito

Quando $E = 255$ e $f \neq 0 \rightarrow \text{NaN}$

Arredondamento para o valor mais próximo

parte inteira par ($n0.$):

parte fracionária $\leq .10 \rightarrow n0$

parte fracionária $> .10 \rightarrow n1$

parte inteira ímpar ($n1.$):

parte fracionária $< .10 \rightarrow n1$

parte fracionária $\geq .10 \rightarrow n1 + 1$

Multiplicação

1) Somar expoentes

2) Multiplicar mantissas

3) Normalizar o resultado

4) Arredondar e renormalizar (se necessário)

$$\text{Ex: } (1.1100 \times 2^0) \cdot (1.1001 \times 2^{-2})$$

$$1) 0 + (-2) = -2$$

$$2) 1.1100 \times 1.1001 = 10.101111$$

Norma IEEE 754 – Multiplicação

multiplicar mantissas e somar expoentes

Exemplo: $N = (1.1100 \times 2^0) \times (1.1001 \times 2^{-2})$

1º Passo: Somar os expoentes

$$\text{Exp. Resultado} = 0 + (-2) = -2$$

$$\begin{array}{r} 1.1001 \\ \times 1.11 \\ \hline 11001 \\ +11001 \\ \hline 10,101111 \end{array}$$

2º Passo: Multiplicar as mantissas

$$M_r = 1.1100 \times 1.1001 = 10.101111$$

$$\begin{array}{r} 11001 \\ +11001 \\ \hline 10,101111 \end{array}$$

3º Passo: Normalizar o resultado

$$N = 10.101111 \times 2^{-2} = 1.0101111 \times 2^{-1}$$

4º Passo: Arredondar o resultado e renormalizar (se necessário)

$$N = 1.0101111 \times 2^{-1} = 1.0110 \times 2^{-1}$$

$$\begin{array}{r} 1.0101 \mid 111 \\ + 0.0000 \mid 100 \\ \hline 1.0110 \mid 011 \end{array}$$

Somamos 1 a direita do ponto e truncamos se ficar não normalizado $\rightarrow$ normalizamos dev mas

Exemplo com 4 bits fracionários

truncamos

Aula 7 - 14 e paramos aí

$$3) 10.101111 \times 2^{-2} = 1.0101111 \times 2^{-1}$$

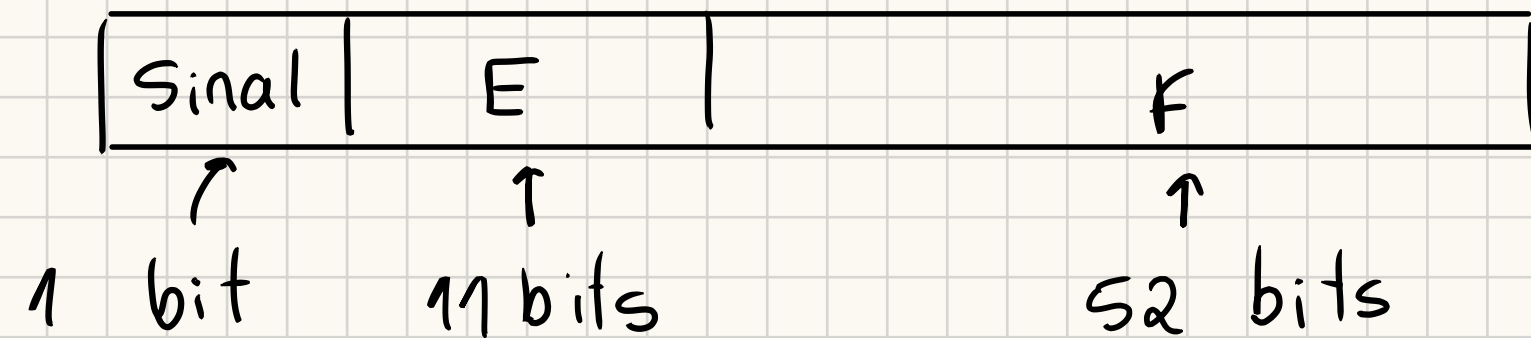
$$4) 1.0101111 = 1.0110$$

$$\begin{array}{r} 1.010111 \\ + 0.00010 \\ \hline 1.01101 \end{array}$$

$$\begin{array}{r} 1.1001 \\ \times 1.11 \\ \hline 11001 \\ 11001 \\ + 11001 \\ \hline 10.101111 \end{array}$$

Precisão Dupla

- maior gama e precisão → tipo double



$$N = (-1)^S 1.f \times 2^{(E-1023)}$$

- Expoente toma valores entre -1022 e +1023 (códigos 1 a 2046 - 0 e 2047 são reservados)
- Representação decimal tem, no máximo, 15 casas decimais

Casos Particulares da norma IEEE 754

Em precisão Simples/Dupla

Expoente	Parte Fracionária	Quando acontece	Representa
0	0		0
0	$\neq 0$	Quando se quer representar valores ainda mais pequenos	Quantidade desnormalizada
255/2047	0	divisão por 0 ex: $1.0/0.0$, $-1.0/0.0$	(+/-) Infinito
255/2047	$\neq 0$	Resultados não numéricos ex: $0.0/0.0$, inf/inf , $\text{NaN} \times 2$	NaN (Not a number)

Modelo de von Neumann

- Existe apenas uma memória partilhada por instruções e dados
- Não é possível aceder a dados e instruções ao mesmo tempo

Modelo de Harvard

- Duas memórias independentes - uma para dados e outra para instruções
- É possível aceder às duas memórias simultaneamente (mais eficaz)
- As duas memórias podem ter dimensões diferentes

Datapath Single Cycle

No MIPS, para a maioria das instruções, as duas primeiras operações são:

- 1) Incrementar o Program Counter (PC) e efetuar essa leitura - **Instruction Fetch**
- 2) Ler dois registos internos (rs e rt)

↑ Em instruções de transferência memória → registo (lw) e instruções com constantes (addi) só é necessário ler o conteúdo do campo rs

- Na instrução "jump" esta leitura não se aplica

Em seguida, para todas as instruções exceto salto incondicional, é utilizada a ALU:

- aritméticas e lógicas para executar a instrução
- instruções de acesso à memória para calcular o endereço a acessar
- instruções de branch para comparação dos registros (ver se é branch taken ou não)

Para "jump" altera-se o valor do PC através do seu código máquina, não sendo necessária recorrer à ALU.
Depois de utilizar a ALU:

- instruções aritméticas e lógicas armazenam o resultado no registro destino
- "sw" acessa a memória para escrita do valor do registro do campo "rt"
- "lw" acessa a memória para leitura e de seguida o valor é escrito no registro destino
- "branch" tem que alterar o valor do PC caso a condição testada seja verdadeira

Tempo de execução das Instruções

A frequência máxima do relógio do datapath single-cycle está limitada pelo tempo de execução da instrução mais longa.
tempo de execução de uma instrução = somatório dos atrasos introduzidos

Por exemplo

- tempo de execução de instruções:

tipo R : 15 ns

SW : 17 ns

LW : 20 ns

BEQ : 13 ns

J : 7 ns

período do relógio → 20 ns

$$f_{\max} = \frac{1}{20} = 50 \text{ MHz}$$

Tempo de execução de um programa = # instruções × CPI × Clock-Cycle

Com CPI (Cycles per instruction) = 1 em single cycle:

$$T_{exec} = \#instr \times Clock_Cycle$$

e

$$Desempenho = 1/T_{exec}$$

sendo assim possível comparar um $CPU_{analise}$ com outro $CPU_{referencia}$:

$$\frac{Desempenho_{CPU-Analise}}{Desempenho_{CPU-Referencia}} = \frac{T_{exec\ CPU-Referencia}}{T_{exec\ CPU-Analise}} \quad (se < 1 \rightarrow CPU_{analise} \text{ é melhor})$$

Pipeline

Instruction Fetch - IF

- Acesso à memória para leitura da próxima instrução $\rightarrow$ PC a endereçar a próxima instrução ($PC + 4$)

Operand Fetch - ID

- Ler registos e decodificar a instrução (em paralelo)

Execute - EX / ALU

- Executar a operação ou calcular um endereço

Memory access - MEM

- Aceder à memória de dados para leitura ou escrita

Write - Back - WB

- Escrever o resultado no registo destino

Tempo de Execução

Num datapath pipeline o período de relógio é o atraso de propagação mais lento

		IF	ID	EX	MEM	WB
Ex:	add \$1, \$2, \$3	add				
	lw \$2, 0(\$4)	lw	add			
	sub \$3, \$4, \$3	sub	lw	add		
	addi \$4, \$4, 4	addi	sub	lw	add	
	and \$5, \$1, \$5	and	addi	sub	lw	add
	sw \$2, 0(\$1)	sw	and	addi	sub	lw

Tempo de execução = N° de fases + (N° de instruções - 1) = 5 + 5 = 10 ciclos de relógio

Hazards

- **Estruturais** - Quando mais do que uma instrução precisa de acessar ao mesmo hardware (não se aplica quando há mais do que 1 memória - modelo Harvard)
- **de Controle** - Quando é necessário fazer IF de uma nova instrução mas existe no pipeline uma instrução que pode alterar o fluxo de execução mas ainda não terminou

Soluções: 1) **stalling da próxima instrução** até saber o resultado do branch

2) previsão estática not taken → executa-se a próxima instrução antes de saber o resultado do branch e se o branch for taken anula-se o que foi lido (com NOP)

Solução usada no MIPS 3) **delayed branch** → executa-se sempre a instrução que se segue ao branch. Isto é feito ao reorganizar as instruções (trocar o branch com a seguinte)

- **de Dados** - Quando existe dependência entre o resultado de uma instrução e o operando de outra mais recente

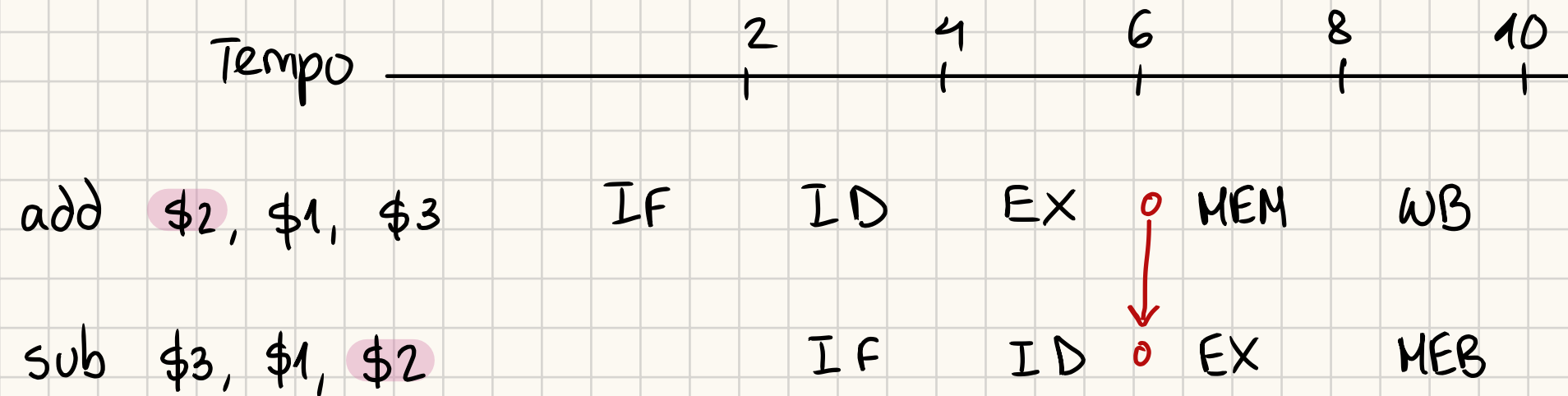
Soluções: 1) **Stall do pipeline** da instrução que necessita do valor, no estágio ID, até a instrução que produz o resultado chegar a WB

Se a escrita no banco de registros for feita meio do ciclo de relógio então a instrução que necessita do valor pode avançar na transição do relógio seguinte

2) Forwarding - Para instruções tipo R, disponibilizar o resultado assim que ele seja calculado na ALU (estágio EX)

Ex:

Para estes graficos
fazem
sentido:
Lên verticalmente!

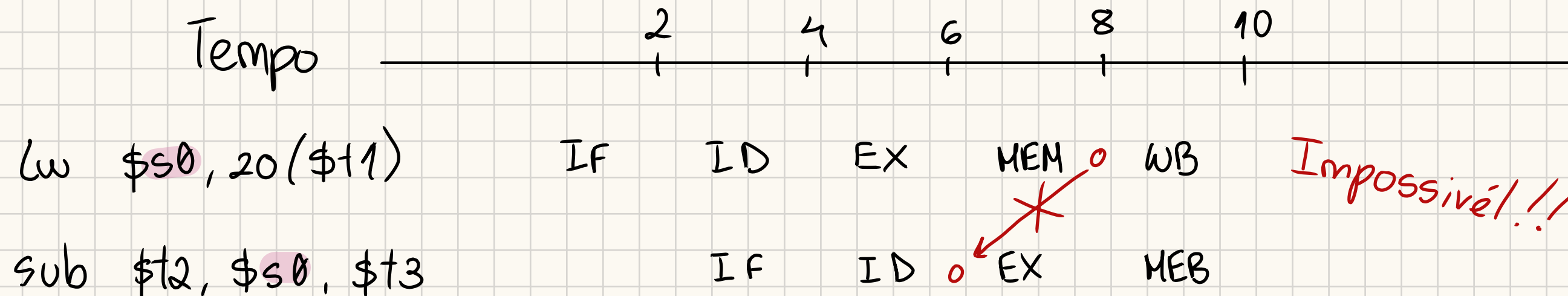


Como o sub precisa do valor de \$2 em EX (estágio em que a operação é efetuada na ALU) mas o add altera o valor de \$2, então a situação pode ser resolvida com forwarding de EX/MEM para ID/EX

Visto que a primeira instrução é do tipo R, o resultado estará pronto quando ela termina a passagem pelo estágio EX.

Quando temos uma instrução desse tipo mas que depende de uma instrução de acesso à memória (LW) o resultado só estará pronto quando LW acabar o estágio MEM

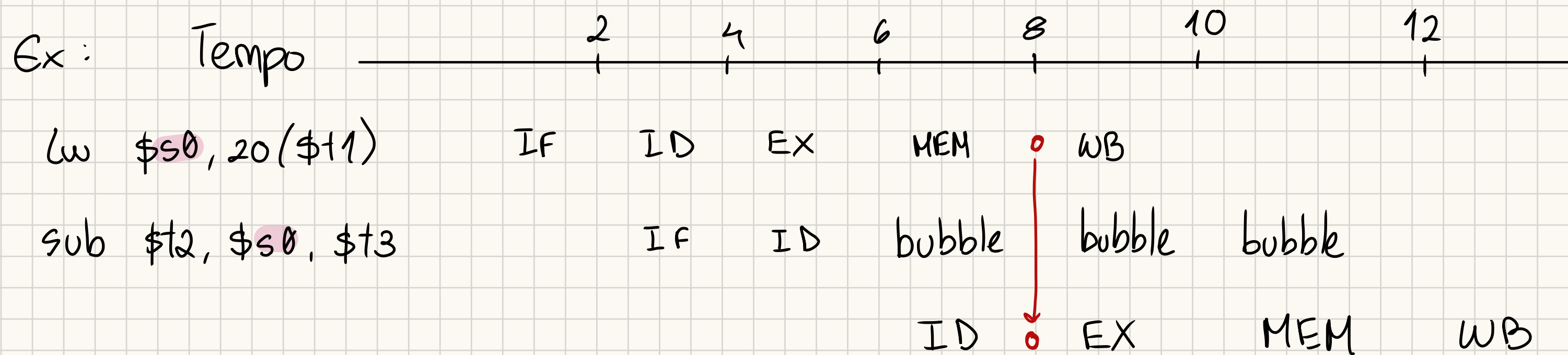
Ex:



Isto seria impossível pois implica enviar informação para trás no tempo (de 10 para 8)
Sendo assim, quando existe uma dependência entre uma instrução de acesso à memória seguida de uma instrução aritmética a solução é:

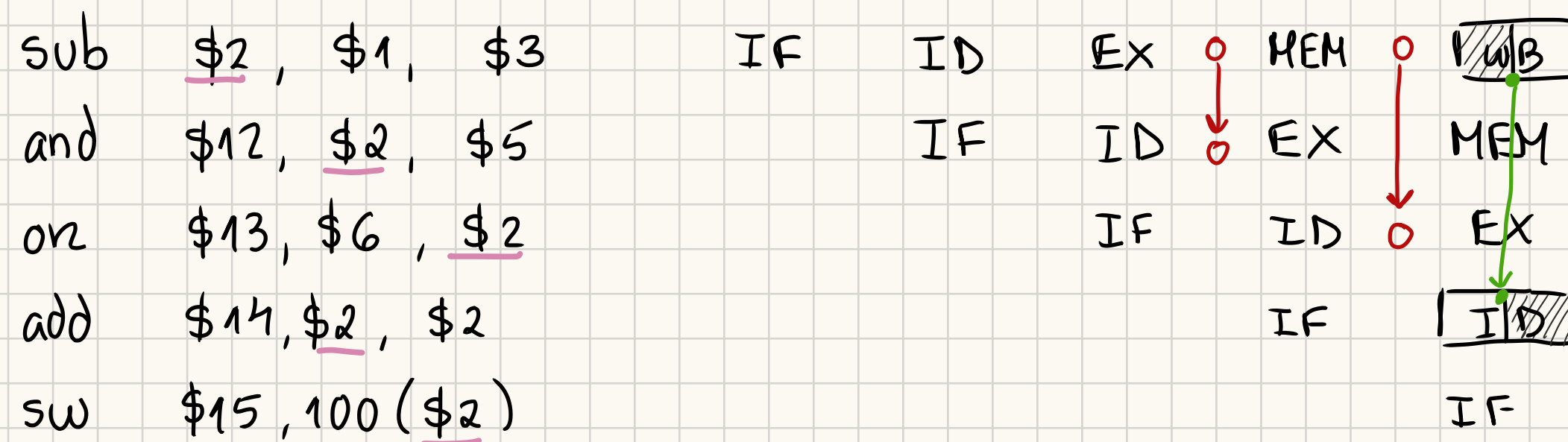
1) fazer stall durante 1 ciclo de relógio

2) Fazer forwarding de MEM/WB para EX (para a entrada da ALU)



Caso não houvesse forwarding resolveria-se com stall de 2 ciclos de relógio

Exemplo de Hazards de dados com forwarding



* momento em que o valor de \$2 já estará calculado

* Fazendo a escrita (WB) a meio do ciclo do relógio juntamente com a leitura (ID) assíncrona permite-nos resolver o 3º Hazard encontrado

Resolvido sem Forwarding!!
Não se considera um Hazard!!

Resumindo Hazards de dados

- Encaminhar o valor que se encontra em estágios mais avançados do pipeline (mas que ainda não foi escrito no registo destino) para onde é necessário
- Instruções branch necessitam de forwarding EX/MEM → ID (mas não MEM/WB → ID)
- As restantes instruções necessitam dos valores corretos na fase EX (forwarding para a ALU)

Para detectar verificamos se existem instruções na fase MEM ou WB cujo registro destino é um operando de uma instrução que esteja na fase EX, ou seja:

$$EX/MEM.RDD == ID/EX.RS \text{ (ou RT)} \quad \text{ou} \quad MEM/WB.RDD == ID/EXX.RT \text{ (ou RS)}$$

$$\text{E } \underline{\text{Regwrite}} == 1!$$

Como fazer stalling?

- 1) Inserir bubble na etapa EX: fazer reset síncrono do registro ID/EX
- 2) Congelar, durante 1 ciclo, as etapas IF e ID e impedir a atualização do PC

Situações particulares

[add \$1, \$2, \$3
beq \$1, \$5, lab → Stall 1T + FW EX/MEM → ID (branch é resolvido em ID)

[lw \$1, 0(\$5)
beq \$1, \$2, lab → Stall 2T (lw só tem o valor correto em MEM)

[lw \$1, 0(\$5)
sw \$1, 4(\$4) → FW MEM/WB → MEM (sw só precisa de \$1 em MEM)

Exercícios

1) Numero de ciclos de relógio?

add \$1, \$2, \$3

lw \$2, 0(\$4)

sub \$3, \$4, \$3

addi \$4, \$4, 4

and \$5, \$1, \$5

sw \$2, 0(\$1)

IF

ID

EX

MEM

WB

IF

ID

EX

MEM

WB

IF

n há hazards $\rightarrow 5 + (6 - 1) = 107$

2) Hazards e como resolver sem forwarding? E com forwarding para EX e ID?

1 main: lw \$1, 0(\$0)

2 add \$4, 0, \$0

3 lw \$2, 4(\$0)

4 loop: lw \$3, 0(\$1)

5 add \$4, \$4, \$3

6 sw \$4, 36(\$1)

7 addiu \$1, \$1, 4

8 sltu \$5, \$1, \$2

9 bne \$5, \$0, loop

10 sw \$4, 8(\$0)

11 lw \$1, 12(\$0)

a) 4 Hazards de dados $\rightarrow$ 2 Stalls em cada e um de controle

b)

EX

ID EX

ID

EX

ID EX

ID

lw só está completo em MEM $\rightarrow$ 1 stall + FW WB $\rightarrow$ EX

FW EX/MEM $\rightarrow$ EX

FW EX/MEM $\rightarrow$ EX

branch precisa do valor em ID $\rightarrow$ 1 stall + FW EX/MEM $\rightarrow$ ID